CSED233 · 데이터구조 · 포스텍

Ch6. Trees (2)

1. DFS vs BFS 복습

두 순회 방식

- **DFS (Depth-First)**: 깊이 먼저 → **Stack(스택)** 기반 (recursion = call stack)
- **BFS (Breadth-First)**: 같은 level 먼저 → **Queue(큐)** 기반 (level-order)
- 둘 다 $O(n)$ (모든 노드 1회 방문)

Pseudocode(의사코드)

DFS (iterative)

```
stack.push(root)
while stack not empty:
    cur = stack.pop()
    visit(cur)
    for each child of cur:
        stack.push(child)
```

BFS (level-order)

```
queue.enqueue(root)
while queue not empty:
    cur = queue.dequeue()
    visit(cur)
    for each child of cur:
        queue.enqueue(child)
```

- **핵심 차이**: Stack → LIFO → 깊이 방향 / Queue → FIFO → 너비 방향

2. Tree Reconstruction (재구성)

질문

- 순회 결과만 보고 원래 tree 구조를 재구성할 수 있는가?

결론

- **전제**: 모든 노드의 값이 서로 다를 때 (중복 값이면 root 위치를 유일하게 못 정함)
- **단일 순회 불가**: preorder 하나, inorder 하나, postorder 하나만으론 tree 고유 식별 불가
- **2개 순회 필요** (단, inorder 반드시 포함):
  - ✓ (preorder + inorder)
  - ✓ (postorder + inorder)
  - ✓ (level-order + inorder)
  - ✗ (preorder + postorder): 모호성 남음

Why inorder 필수?

- **inorder**: root 기준으로 **left subtree / right subtree 분할** 가능
- **preorder / postorder / level-order**: **root 위치** 찾기 가능
  - preorder: 맨 앞
  - postorder: 맨 뒤
  - level-order: 맨 앞 (level 0)
- → 'root 식별 + 좌우 분할'이 함께 있어야 재귀적으로 재구성

알고리즘 (Preorder + Inorder)

```
reconstruct(pre, in):
  (1) Root = pre[0]                    ← preorder 첫 원소
  (2) inorder에서 Root 위치 찾기 → 좌/우 분할
      Left_subtree  = Root 왼쪽 원소들
      Right_subtree = Root 오른쪽 원소들
  (3) Left_child  = pre에서 Left_subtree 노드 중 가장 먼저 나오는 것
      Right_child = pre에서 Right_subtree 노드 중 가장 먼저 나오는 것
  (4) 각 subtree에 대해 재귀
```

예시

- Inorder(중위 순회): ⟨20, 30, 35, 40, 45, 50, 55, 60, 70⟩
- Postorder(후위 순회): ⟨20, 35, 30, 45, 40, 55, 70, 60, 50⟩
- → Root = 50 (postorder 끝)
- → Left: {20,30,35,40,45}, Right: {55,60,70}
- → 재귀 반복하여 재구성

3. General k-ary Tree

정의

- **k-ary tree**: 각 노드의 자식 수가 최대 k개
- **일반 tree**: 자식 수 제한 없음, 순서 없을 수도 있음
- Binary tree와 달리 **자식 개수가 가변** → 구현 방식이 다양

대표 구현 3가지

1. **Parent-pointer array** (simple approach)
2. **List-of-Children**
3. **Left-Child / Right-Sibling**

   각각 pros/cons 있음 → 연산 빈도에 따라 선택.

4. Simple Approach (Parent-Pointer)

구조

- 두 병렬 배열:
  - `Tree[k] = m` → 노드 k의 parent가 노드 m
  - `Label[k]` → 노드 k의 value
- Root는 `Tree[root] = 0` (convention)

시간복잡도

연산	복잡도	이유
`Parent(k, T)`	$O(1)$	`Tree[k]` 한번 접근
`Leftmost_Child(k, T)`	$O(n)$	전체 배열 스캔 필요 (children 정보 없음)
`Create(v, T₁,...,Tₘ)`	$O(n)$	**모든 subtree index 재배치 필요**

Create 연산의 어려움

- 두 tree를 합치려면 두 번째 tree의 **모든 index를 shift** 해야 함
- 예: T_1이 6 노드, T_2가 3 노드 → T_2의 index 1,2,3 → 8,9,10으로 변경 + parent 값도 전부 수정
- → $O(n)$ 작업

5. List-of-Children Approach

아이디어

- Parent info **+ children info 동시 유지**
- 각 non-terminal 노드에 대해 **linked list of children** 추가

Combining Trees: $C_2(v, T_1, T_2)$

- **단일 배열 저장 시**: 두 tree가 한 배열에 있으면 → parent pointer만 바꾸면 됨 → $O(1)$
  - binary tree 한정: 자식 2개이므로 새 root v의 children list 만들기도 상수
- **분리 배열 저장 시**: 한 배열로 합치려면 index 재배치 → $O(n)$

문제점

1. **Right sibling 접근 어려움**: y의 sibling을 찾으려면 → parent 찾고 → parent의 children list 뒤져야 함
2. **데이터 중복**: 각 노드가 node array와 parent의 children list에 **두 번 등장**

6. Left-Child / Right-Sibling (LC/RS)

아이디어

- 각 노드에 정확히 **2개의 pointer**만 저장:
  - `Lc` : leftmost child
  - `Rs` : right sibling
- 추가로 `Pa` (parent)도 저장 가능
- **자식 수 무관** → 어떤 k-ary tree든 2-pointer로 표현

핵심 통찰

어떤 general k-ary tree도 binary tree로 변환 가능!

- 이유: (leftmost-child, right-sibling) 두 링크만 있으면 구조 재구성 가능 → 구조적으로 binary tree와 동형

Dynamic (Linked) 구현

- array 고정 크기 문제 → linked list로:

```
struct node {
   char label;
   node* lc;   // leftmost child (= binary's left)
   node* rs;   // right sibling  (= binary's right)
};
```

- → **사실상 binary tree node 구조**

7. Knuth Transform (변환 절차)

3단계

1. **Leftmost child → Left child**
2. **Right sibling → Right child** (단, tree root들은 서로 sibling으로 간주 — forest 처리)
3. **시계방향 45° 회전** → 시각적으로 binary tree 형태 완성

순회 결과 비교

순회	Original general tree	Converted binary tree
Preorder(전위 순회)	`a b d e f c`	`a b d e f c` ✓ **동일**
Inorder	`d b e f a c`	`d e f b c a` ✗ 다름
Postorder	`d e f b c a`	`f e d c b a` ✗ 다름

- Original 열의 inorder convention: 첫째 자식 → root → 나머지 자식 (general tree 표준)
- **핵심**: **Preorder는 변환 전후 동일!**
- 이유: leftmost-child → left child 매핑이 'root 먼저 → 왼쪽 먼저' 순서를 보존

Q1. Converted binary tree를 general tree로 역변환하려면?

> **힌트**

LCRS 링크의 의미를 뒤집는다: left = 첫째 자식, right = 다음 형제.

> **정답**

개념은 힌트 참고.

반대 과정. root의 left child → leftmost child, 그 노드의 right chain → sibling들. 시계방향 반대로 45° 회전.

8. 구현 비교 정리

시간복잡도 요약

연산	Simple (Parent)	List-of-Children	LC/RS
Parent(k)	$O(1)$	$O(1)$	$O(1)$
Leftmost child	$O(n)$	$O(1)$	$O(1)$
Right sibling	$O(n)$	$O(\deg)$	$O(1)$

Create (combine)	$O(n)$	$O(1)$ (single array)	$O(1)$
공간 효율	좋음	중복 발생	좋음

선택 기준

- **Parent 조회 위주** → Simple
- **Children 순회 위주** → List-of-Children
- **일반 tree를 binary tree처럼 다루고 싶다** → LC/RS (+ Knuth transform)

Summary

- DFS = stack, BFS = queue, 둘 다 $O(n)$
- 재구성: inorder + (pre/post/level) 중 하나 필요
- General tree 구현 3가지: Parent / List-of-Children / LC-RS
- **Knuth transform**: general tree → binary tree (preorder 보존)